COS 433: Cryptography

Announcements:

- Problem Set 3 due **10/28**

- Lecture notes on computational number theory (by Dana Angluin) on the course website

Public-Key Encryption

Public-Key Encryption



$m \in \{0,1\}^\lambda$

pk (broadcast)

$\mathrm{ct} = \mathrm{Enc}(\mathrm{pk}, m; r)$



$(\mathrm{pk}, \mathrm{sk}) \leftarrow \mathrm{Gen}(1^\lambda)$

$m' = \mathrm{Dec}(\mathrm{sk}, \mathrm{ct})$

Definition: A public-key encryption scheme consists of algorithms $(\mathbf{Gen}, \mathbf{Enc}, \mathbf{Dec})$. It must satisfy the following properties:

- Efficiency: $\mathbf{Gen}, \mathbf{Enc}, \mathbf{Dec}$ run in polynomial time
- Correctness: $\mathrm{Dec}(\mathrm{sk}, \mathrm{Enc}(\mathrm{pk}, m)) = m$ for all $(\mathrm{pk}, \mathrm{sk}) \leftarrow \mathrm{Gen}(1^\lambda)$.
- Computational security: For all m_0, m_1, $(\mathrm{pk}, \mathrm{Enc}(\mathrm{pk}, m_0)) \approx_c (\mathrm{pk}, \mathrm{Enc}(\mathrm{pk}, m_1))$

CPA Security for PKE (equivalent)

Challenger



pk

$b \leftarrow \{0,1\}$

pk

$m_0^{(\lambda)}, m_1^{(\lambda)}$

$\mathrm{Enc}(\mathrm{pk}, m_b; r)$

Adversary



b'

Win if $b' = b$

Definition: A public-key encryption scheme with message length $n = n(\lambda)$ satisfies IND-CPA security if

For all polynomial-time  ,

$$\Pr[\mathrm{Win}] \leq \frac{1}{2} + \mathrm{negl}(\lambda)$$

Public-Key Encryption



$m \in \{0,1\}^n$

pk (broadcast) ←

$\text{ct} = \text{Enc}(\text{pk}, m; r)$ →



$(\text{pk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda)$

$m' = \text{Dec}(\text{sk}, \text{ct})$

Lemma: Suppose that $(\text{Gen}, \text{Enc}, \text{Dec})$ is a public-key encryption scheme with messages of length $n = 1$. Then, there exists a PKE with message length $n(\lambda)$ for all polynomial functions $n(\lambda)$, satisfying CPA-security.

Proof: encrypt bit-by-bit. Security holds by a hybrid argument.

Next: RSA Encryption and Diffie-Hellman

RSA Computational Problem

How hard is it to solve equations in one variable over $\mathbb{Z}_N$?

As hard as factoring: Given a, find x such that $x^2 \equiv a \pmod{N}$

QR (believed hard): Given a, decide if $x^2 \equiv a \pmod{N}$ has a solution

What about equations of higher degree?

RSA Computational Problem

How hard is it to solve equations in one variable over $\mathbb{Z}_N$?

Cube root finding (believed hard): Given a, find x such that $x^3 \equiv a \pmod{N}$

Candidate OWF:

$$f(p, q, x) = (pq, x^3 \bmod pq)$$

Cube root mod N assumption

Assumption:

For all poly-time A,

$$\Pr_{\substack{p,q \leftarrow \{0,1\}^\lambda \\ p,q \text{ prime}, N = pq \\ x \leftarrow \mathbb{Z}_N^\times}} [A(N, x^3 \bmod N) = y : y^3 \equiv x^3 \bmod N] = \text{negl}(\lambda)$$

RSA Computational Problem

Cube root finding (believed hard): Given a, find x such that $x^3 \equiv a \pmod{N}$

How does this compare to square root finding? Depends on p, q

Tool: Euler's theorem. For all $a \in \mathbb{Z}_N^\times$ $(\gcd(a, N) = 1)$,

$$a^{\phi(N)} \equiv 1 \pmod{N}$$

where $\phi(N) = (p-1)(q-1) = |\mathbb{Z}_N^\times|$.

RSA Computational Problem

Cube root finding (believed hard): Given a, find x such that $x^3 \equiv a \pmod{N}$

How does this compare to square root finding? Depends on p, q

Tool: Euler's theorem. For all $a \in \mathbb{Z}_N^\times$ $(\gcd(a, N) = 1)$,

$$a^{\phi(N)} \equiv 1 \pmod{N}$$

Case 1: if 3 divides $\phi(N)$, then $x \mapsto x^3$ is many-to-one, and cube root-finding is as hard as factoring! (But it's not too different from square root finding.)

RSA Computational Problem

Cube root finding (believed hard): Given a, find x such that $x^3 \equiv a \pmod{N}$

How does this compare to square root finding? Depends on p, q

Tool: Euler's theorem. For all $a \in \mathbb{Z}_N^\times$ $(\gcd(a, N) = 1)$,

$$a^{\phi(N)} \equiv 1 \pmod{N}$$

Case 2: if 3 does not divide $\phi(N)$, then $x \mapsto x^3$ is a permutation on $\mathbb{Z}_N^\times$!

Reason: $\exists d: 3d \equiv 1 \pmod{\phi(N)}$. $y \mapsto y^d$ is an inverse map!

RSA Computational Problem

Cube root finding (believed hard): Given a, find x such that $x^3 \equiv a \pmod{N}$

How does this compare to square root finding? Depends on p, q

Tool: Euler's theorem. For all $a \in \mathbb{Z}_N^\times$ $(\gcd(a, N) = 1)$,

$$a^{\phi(N)} \equiv 1 \pmod{N}$$

Case 2: if 3 does not divide $\phi(N)$, then $x \mapsto x^3$ is a permutation on $\mathbb{Z}_N^\times$!

But: no longer know that cube root finding is as hard as factoring.

RSA Computational Problem

Candidate OWF:

$$f_N(x) = x^3 \bmod N$$

When 3 does not divide $(p-1)(q-1)$, this is a Trapdoor Permutation

- One-way permutation (defined on $\mathbb{Z}_N^\times$)
- Given trapdoor d, permutation becomes easy to invert!
- “RSA with exponent 3"

RSA Computational Problem

Another candidate trapdoor permutation:

$$f_{N,e}(x) = x^e \bmod N$$

where $\gcd(e, \phi(N)) = 1$. Typical: sample $e \leftarrow \mathbb{Z}^{\times}_{\phi(N)}$.

- "RSA trapdoor permutation"
- Given p, q, can find trapdoor d: $e \cdot d \equiv 1 \pmod{\phi(N)}$.
- $(x^e)^d \equiv x^{ed} \equiv x^1 \equiv x \bmod N$

RSA Assumption

Assumption:

For all poly-time A,

$$\Pr_{\substack{p,q \leftarrow \{0,1\}^\lambda \\ p,q \text{ prime}, N = pq \\ e \leftarrow \mathbb{Z}^\times_{\phi(N)} \\ x \leftarrow \mathbb{Z}^\times_N}} [A(N, e, x^e \bmod N) = x] = \text{negl}(\lambda)$$

Open: prove factoring assumption $\Leftrightarrow$ RSA assumption

RSA Assumption

Assumption:

For all poly-time A,

$$\Pr_{\substack{p,q \leftarrow \{0,1\}^\lambda \\ p,q \text{ prime}, N=pq \\ e \leftarrow \mathbb{Z}^\times_{\phi(N)} \\ x \leftarrow \mathbb{Z}^\times_N}} [A(N, e, x^e \bmod N) = x] = \text{negl}(\lambda)$$

Exercise: finding inverse $d \Leftrightarrow$ factoring.

“Textbook RSA Encryption”

Heuristic PKE scheme

$$\mathrm{Gen}(1^\lambda) \to (\mathrm{pk} = (N, e), \mathrm{sk} = d)$$
$$\mathrm{Enc}(\mathrm{pk}, m) = m^e \bmod N.$$
$$\mathrm{Dec}(\mathrm{sk}, \mathrm{ct}) = \mathrm{ct}^d \bmod N.$$

- Correct!
- Hard to recover random m given ct
- Deterministic, cannot satisfy any of our security definitions

“Textbook RSA Encryption”

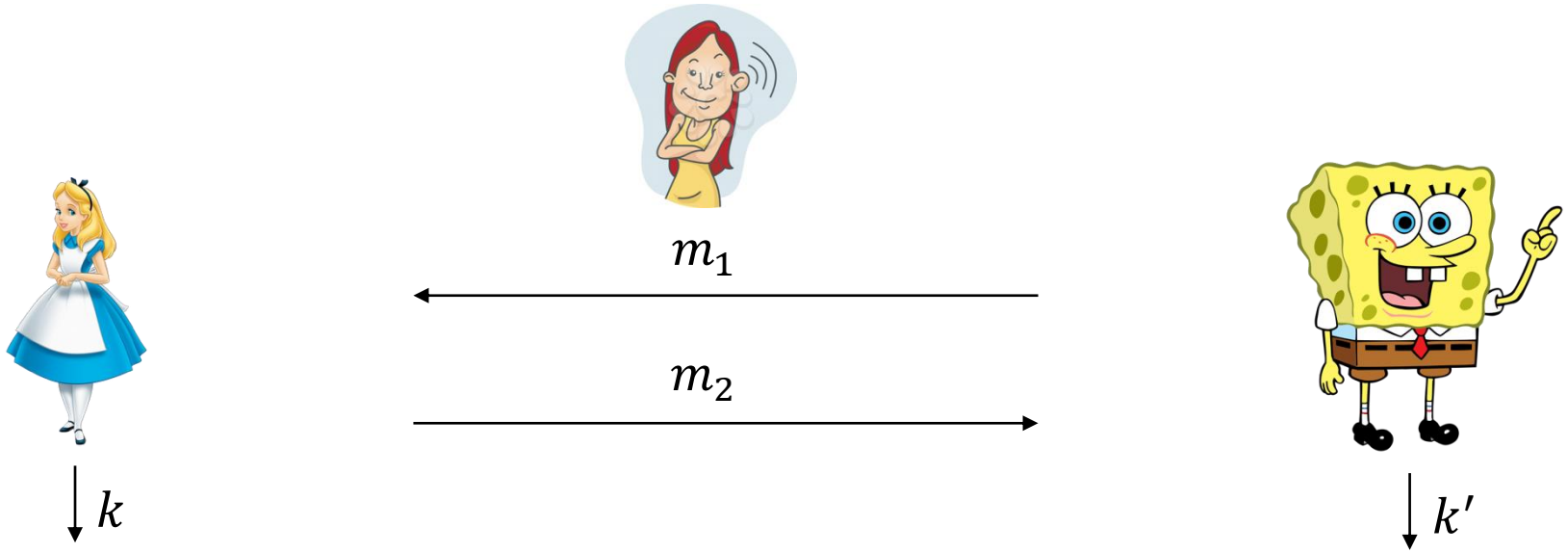
Heuristic PKE scheme

$$\mathrm{Gen}(1^\lambda) \to (\mathrm{pk} = (N, e), \mathrm{sk} = d)$$
$$\mathrm{Enc}(\mathrm{pk}, m) = m^e \bmod N.$$
$$\mathrm{Dec}(\mathrm{sk}, \mathrm{ct}) = \mathrm{ct}^d \bmod N.$$

- Can be turned into a secure PKE in several ways (see HW4 for one way)
- Efficient CCA-secure variant of RSA used in practice (RSA-OAEP)

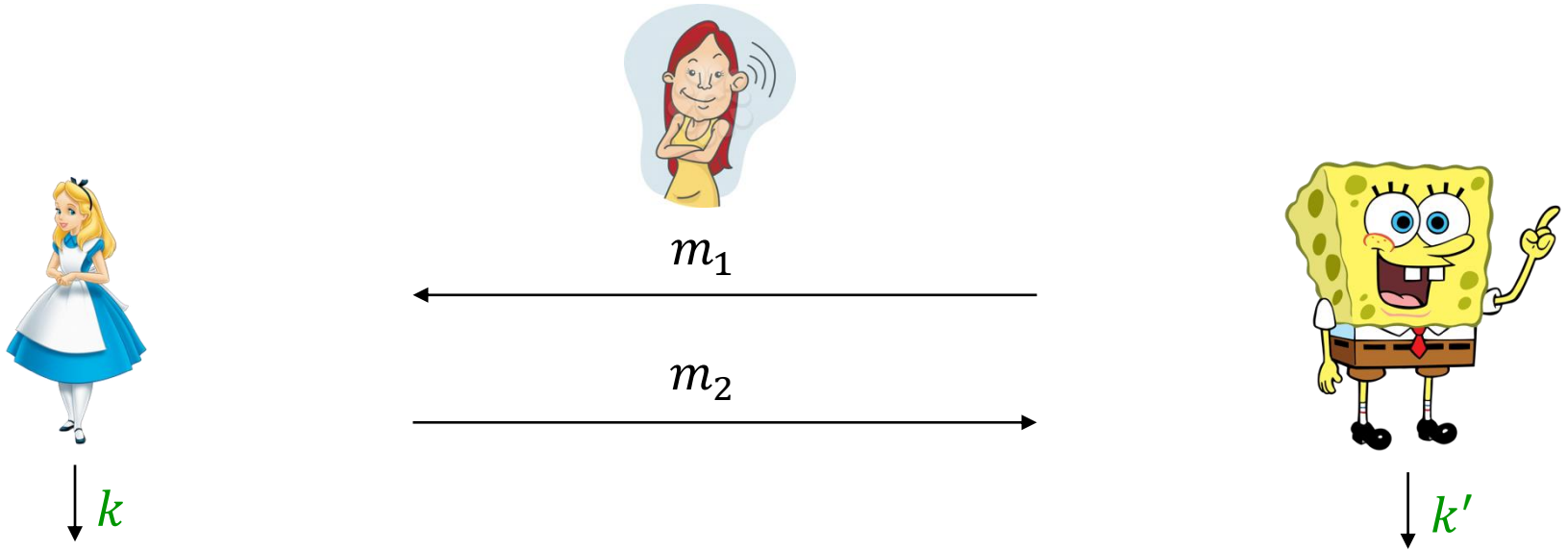
Diffie-Hellman “Key Exchange”

Key Agreement



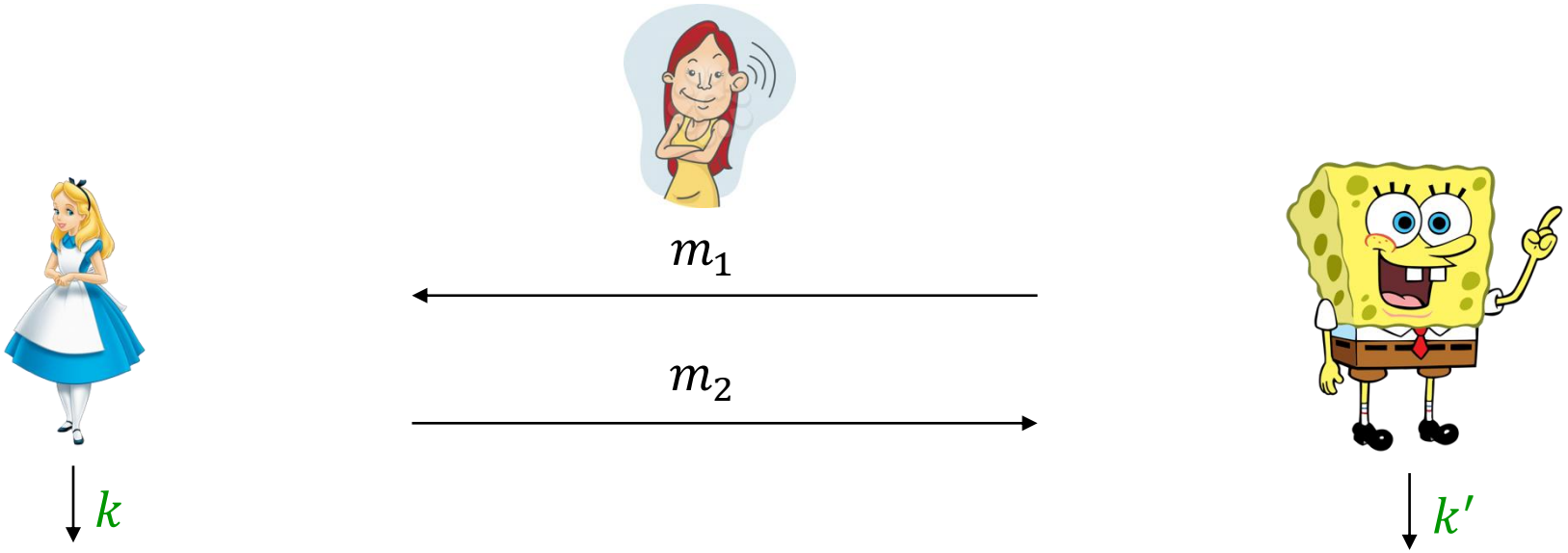
- Goal: share a secret key k that looks uniformly random to any 
- Alice and Bob do not want to meet in person
- They run a protocol (Bob sends a message, then Alice sends a message)

Key Agreement



- Correctness: if Alice and Bob behave honestly, then $k = k'$
- Weak security: hard for  to find k given (m_1, m_2).
- Strong security: $(m_1, m_2, k) \approx_c (m_1, m_2, u)$ (uniformly random key u)

Key Agreement



Can be solved using PKE

Key Agreement





pk

$\mathrm{ct} = \mathrm{Enc}(\mathrm{pk}, k; r)$



$k \leftarrow \{0,1\}^{\lambda}$

$(\mathrm{pk}, \mathrm{sk}) \leftarrow \mathrm{Gen}(1^{\lambda})$

$k' = \mathrm{Dec}(\mathrm{sk}, \mathrm{ct})$

Can be solved using PKE

- Strong security follows from PKE security
- Protocol establishes a session key which can be used to encrypt new messages using SKE (e.g. AES)

The Diffie-Hellman Proposal

Tool: $\mathbb{Z}_p$-discrete logarithm problem

- Hard problem defined over $\mathbb{Z}_p$ instead of $\mathbb{Z}_N$
- Number theory background:
  - $\mathbb{Z}_p^\times = \{1, 2, \ldots, p-1\}$ is a multiplicative group
  - Theorem: $\mathbb{Z}_p^\times$ is cyclic
    - There exists a generator $g \in \mathbb{Z}_p^\times$ such that $\mathbb{Z}_p^\times = \{1, g, g^2, \ldots, g^{p-2}\}$

Discrete log problem: given $(g, g^x \bmod p)$, find x

Discrete log assumption over $\mathbb{Z}_p^\times$

Assumption: parametrized by a prime p and generator g

For all poly-time A,

$$\Pr_{x \leftarrow \mathbb{Z}_{p-1}}[A(p, g, g^x) = x] = \mathrm{negl}(\lambda)$$

- Where do (p, g) come from?
  - p can be sampled uniformly at random from $\{0,1\}^\lambda$ (check primality)
  - Sample g uniformly? Unclear how to tell if g is a generator

Discrete log assumption over $\mathbb{Z}_p^\times$

Assumption: parametrized by a prime p and generator g

For all poly-time A,

$$\Pr_{x \leftarrow \mathbb{Z}_{p-1}}[A(p, g, g^x) = x] = \mathrm{negl}(\lambda)$$

- Where do (p, g) come from?
  - Sample $p - 1$ to be a uniformly random **factored** integer (tricky)
    - Check if p is prime. Else try again.
  - Sample g uniformly and check that $g^{\frac{p-1}{q}} \neq 1$ for all prime $q | p - 1$

Discrete log assumption over $\mathbb{Z}_p^\times$

Assumption (random p, g): for all poly-time A,

$$\Pr_{\substack{p \leftarrow \{0,1\}^\lambda,\ p \text{ prime} \\ g \leftarrow \mathbb{Z}_p^\times,\ g \text{ generator} \\ x \leftarrow \mathbb{Z}_{p-1}}} [A(p, g, g^x) = x] = \text{negl}(\lambda)$$

- Implies the existence of one-way functions
  - $f(p, g, x) = (p, g, g^x)$ (up to domain sampling tricks)
- Not known to imply PKE
- We will make stronger assumptions.

Diffie-Hellman Key Agreement

p, g known (or sampled by Bob)





$m_1 = g^x$

$m_2 = g^y$



$y \leftarrow \mathbb{Z}_{p-1}$

$k = m_1^y$

$x \leftarrow \mathbb{Z}_{p-1}$

$k' = m_2^x$

- Correctness:

$$k = m_1^y \equiv g^{xy} \equiv m_2^x = k'$$

Diffie-Hellman Key Agreement

p, g known (or sampled by Bob)





$y \leftarrow \mathbb{Z}_{p-1}$

$k = m_1^y$

$m_1 = g^x$

$m_2 = g^y$



$x \leftarrow \mathbb{Z}_{p-1}$

$k' = m_2^x$

- Security?
  - If discrete logarithm is easy, then this is broken.

Computational Diffie-Hellman Assumption over $\mathbb{Z}_p^\times$

Assumption parametrized by a prime p and generator g (usually sampled at random): For all poly-time A,

$$\Pr_{x,y \leftarrow \mathbb{Z}_{p-1}}[A(p, g, g^x, g^y) = g^{xy}] = \mathrm{negl}(\lambda)$$

- Implies the weak security of Diffie-Hellman key agreement
- Open: show this is equivalent to the hardness of discrete log

What about strong security?

Decisional Diffie-Hellman Assumption over $\mathbb{Z}_p^\times$

Assumption parametrized by a prime p and generator g (usually sampled at random):

$$(p, g, g^x, g^y, g^{xy}) \approx_c (p, g, g^x, g^y, g^z)$$

- Implies the strong security of Diffie-Hellman key agreement
- False!
- g^{xy} is a square mod p with probability ¾; g^z is with probability ½

Decisional Diffie-Hellman Assumption over $\mathbb{Z}_p^\times$

Assumption parametrized by a prime p and generator g (usually sampled at random):

$$(p, g, g^x, g^y, g^{xy}) \approx_c (p, g, g^x, g^y, g^z)$$

Now what?

Decisional Diffie-Hellman Assumption over G

Assumption: parametrized by a cyclic group G and generator g (usually sampled from a distribution)

$$(\langle G \rangle, g, g^x, g^y, g^{xy}) \approx_c (\langle G \rangle, g, g^x, g^y, g^z)$$

Solution: we don't have to work over $\mathbb{Z}_p$!

Decisional Diffie-Hellman Assumption over G

Assumption: parametrized by a cyclic group G and generator g (usually sampled from a distribution)

$$(\langle G \rangle, g, g^x, g^y, g^{xy}) \approx_c (\langle G \rangle, g, g^x, g^y, g^z)$$

What we need: a "family of effective groups"

- Group $G^{(\lambda)}$ for each λ (or a distribution over groups)
- $\mathrm{poly}(\lambda)$ time group operations
- Easy to sample a group along with a generator g for $G^{(\lambda)}$

Decisional Diffie-Hellman Assumption over G

Assumption: parametrized by a cyclic group G and generator g (usually sampled from a distribution)

$$(\langle G \rangle, g, g^x, g^y, g^{xy}) \approx_c (\langle G \rangle, g, g^x, g^y, g^z)$$

In what kinds of groups is DDH plausible?

- Need discrete log to be hard
- Need G not to have small subgroups.

Decisional Diffie-Hellman Assumption over G

Assumption: parametrized by a cyclic group G and generator g (usually sampled from a distribution)

$$(\langle G \rangle, g, g^x, g^y, g^{xy}) \approx_c (\langle G \rangle, g, g^x, g^y, g^z)$$

In what kinds of groups is DDH plausible?

- Simple idea: focus on G such that $|G| = q$ is a prime!

Decisional Diffie-Hellman Assumption over G

Assumption: parametrized by a cyclic group G and generator g (usually sampled from a distribution)

$$(\langle G \rangle, g, g^x, g^y, g^{xy}) \approx_c (\langle G \rangle, g, g^x, g^y, g^z)$$

In what kinds of groups is DDH plausible?

- QR_p where $\frac{p-1}{2}$ is a prime. ("Safe Primes")
- QR_N where $N = pq$ and p, q are safe primes (not prime order).

Decisional Diffie-Hellman Assumption over G

Assumption: parametrized by a cyclic group G and generator g (usually sampled from a distribution)

$$(\langle G \rangle, g, g^x, g^y, g^{xy}) \approx_c (\langle G \rangle, g, g^x, g^y, g^z)$$

In what kinds of groups is DDH plausible?

- QR_p where $\frac{p-1}{2}$ is a prime. ("Safe Primes")
- (Prime order) "elliptic curve groups"

Diffie-Hellman Key Agreement

$G, L = |G|, g$ known or sampled by Bob 



$$m_1 = g^x \quad \longleftarrow$$

$$m_2 = g^y \quad \longrightarrow$$



$y \leftarrow \mathbb{Z}_L$

$\downarrow$

$k = m_1^y$

$x \leftarrow \mathbb{Z}_L$

$\downarrow$

$k' = m_2^x$

Theorem:

- If CDH is true for $(\langle G \rangle, g)$, then protocol is weakly secure
- If DDH is true for $(\langle G \rangle, g)$, then protocol is strongly secure

Diffie-Hellman Key Agreement

$G, L = |G|, g$ known or sampled by Bob 



$y \leftarrow \mathbb{Z}_L$

$k = m_1^y$

$m_1 = g^x$

$m_2 = g^y$



$x \leftarrow \mathbb{Z}_L$

$k' = m_2^x$

Note: you don't even need to know L to run this (details omitted)

ElGamal Encryption

$G, L = |G|, g$ known or sampled by Bob (part of pk)



h

$\mathrm{ct} = g^y, h^y \cdot m$



$m' = \mathrm{ct}_2 \cdot \mathrm{ct}_1^{-x}$

$m \in \{0,1\}^n \subset G$

$y \leftarrow \mathbb{Z}_L$

$x \leftarrow \mathbb{Z}_L$

$\begin{pmatrix} \mathrm{pk} = h = g^x, \\ \mathrm{sk} = x \end{pmatrix}$

PKE based on the DDH assumption; "encryption version" of Diffie-Hellman protocol

- Correctness: $h^y = (g^y)^x$
- Lemma: if the DDH assumption holds, then ElGamal encryption is secure.
- Note: unlike our other schemes, this does not use a trapdoor permutation

Algorithms for Discrete Log (general groups)

1. Brute force
   - Running time: $|G| \cdot \mathbf{poly}(\lambda) = 2^{\lambda} \cdot \mathbf{poly}(\lambda)$ for $|G| \approx 2^{\lambda}$
2. Baby Step—Giant Step
   - Let $|G| \leq B^2$ (think $B = 2^{\lambda/2}$)
   - Given $(g, h = g^x)$, solve for $0 \leq y, z \leq B$ such that

$$h = g^{y \cdot B + z}$$

   - Enumerate all $g^{y \cdot B}, h \cdot g^{-z}$. Look for a collision!
   - Running time $= B \cdot \mathbf{poly}(\lambda)$. Quadratic speedup ($2^{\lambda/2}$) over brute force.
3. That's all we know!

(Certain) Elliptic curve groups: can set $\lambda = 256$ (pretty small!)

Algorithms for Discrete Log ($\mathbb{Z}_p^\times$ and subgroups)

1. Brute force
   - Running time: $|G| \cdot \mathbf{poly}(\lambda) = 2^\lambda \cdot \mathbf{poly}(\lambda)$ for $|G| \approx 2^\lambda$
2. Baby Step—Giant Step
   - Running time $= B \cdot \mathbf{poly}(\lambda)$. Quadratic speedup ($2^{\lambda/2}$) over brute force.
3. Index Calculus Algorithms
   - Use random-self reduction: $\mathbf{dlog}(g^x) = \mathbf{dlog}(g^{x+r}) - r$
   - Hope that g^{x+r} is "special," develop a tailored algorithm for "special" h
   - Since over $\mathbb{Z}_p^\times$, g^{x+r} is a random integer, one can show it is often special
     - Quadratic Sieve runs in time $2^{O\left(\sqrt{\lambda \cdot \log \lambda}\right)}$
     - Number Field Sieve runs heuristically in time $2^{O\left(\sqrt[3]{\lambda \cdot \log^2(\lambda)}\right)}$

Algorithms for CDH (any group)

1. Solve discrete log over G

2. That's basically all we know!

Algorithms for DDH (any group)

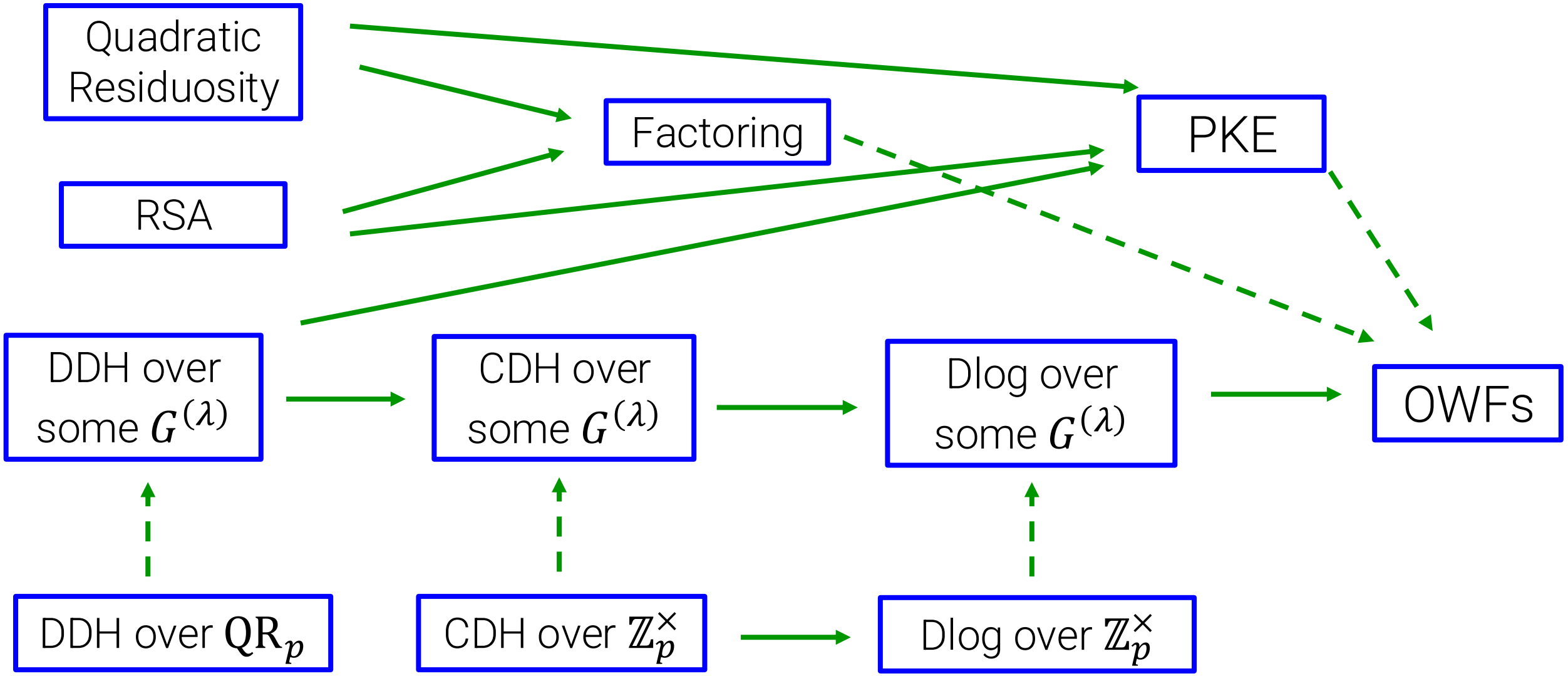
1. Solve discrete log over G

2. Solve discrete log over subgroups of G
   - Breaks certain G such as $\mathbb{Z}_p^\times$

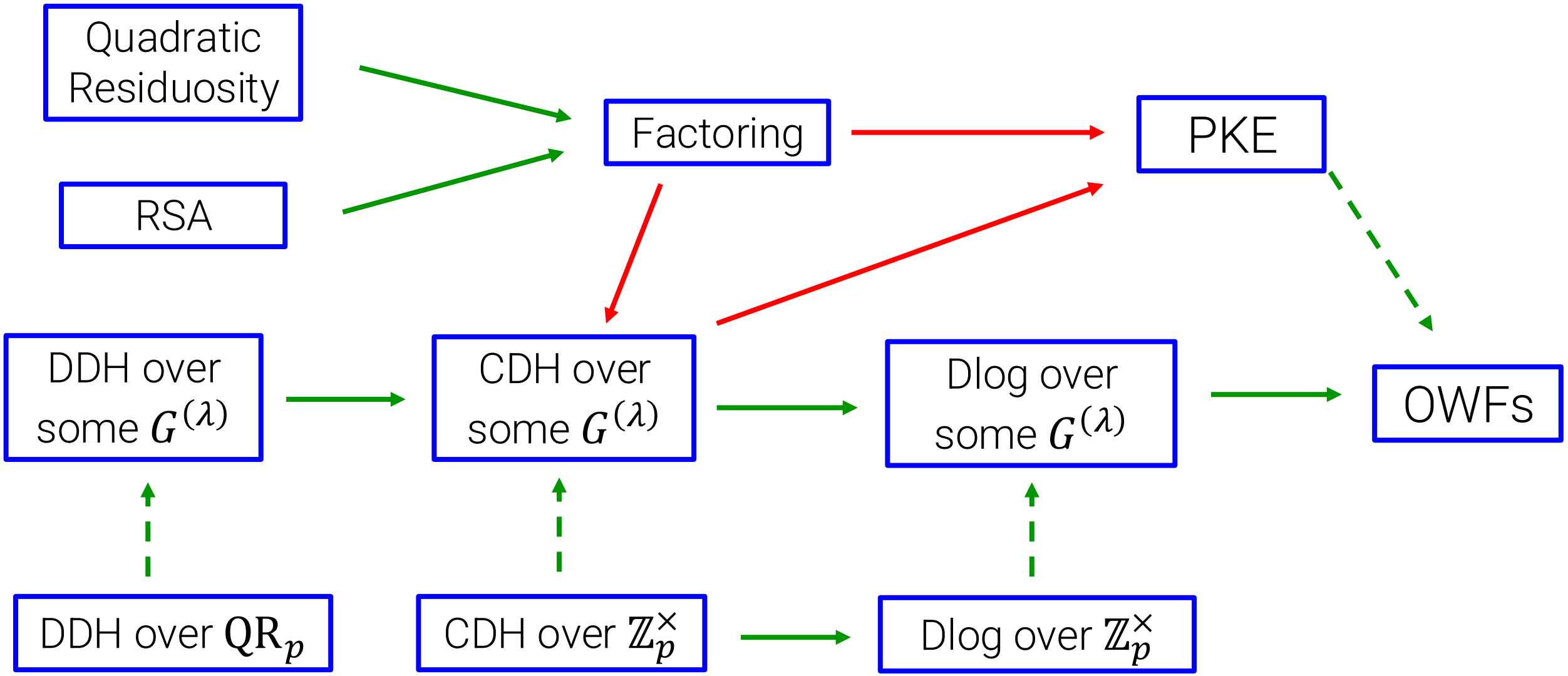
3. It's complicated…
   - DDH broken over certain weird/interesting groups ("self-pairings")

Nevertheless, groups such as QR_p have no known algorithms beyond (1)

Number-Theoretic Hardness Assumptions



Number-Theoretic Hardness Assumptions



End of PKE discussion